# MY TECH PORTFOLIO

Dicle Bulut

More detailed walkthroughs of my personal projects available here.

# PROJECTS

1 Uber Dynamic Pricing Project (DQN)

2 Flight Delay Prediction Project (Random Forest)

2 Earthquake Prediction Project (Markov Chain) (WIP)

# UBER DYNAMIC PRICING PROJECT

VIEW ON GITHUB

# CONTEXT & PROBLEM

| | |
|---|---|
| Context | Ride-sharing platforms operate in a dynamic environment where demand, supply, and customer behaviour fluctuate constantly. Understanding how different pricing strategies perform against each other is critical for competitive advantage in this space. |
| Problem | Traditional Static Pricing (Rules Based) vs Machine Learning Driven Dynamic Pricing: which one yields more revenue and acceptance rate? |

# APPROACH

## Data

Real NYC Uber data from 2021, incorporating practical factors like trip distance, weather conditions, time of day, neighbourhoods, and customer behaviour signals (Source: Kaggle)

## Static

Learns one optimal price-per-mile from historical data and applies it uniformly across all rides.

Models customer acceptance using a price sensitivity curve: the more you deviate from the base fare, the fewer customers accept.

## Dynamic

Learns context-dependent price multipliers (0.8x–1.2x) for each trip using a 4-layer neural network trained on trip features via experience replay and epsilon-greedy exploration.

Selects the best multiplier from 11 options based on what maximises profit for that specific trip context.

Uses the identical sigmoid-based price-sensitivity curve, but implicitly learns which trips tolerate premium pricing

# WILLINGNESS-TO-PAY

A sigmoid-based price acceptance model that maps the absolute deviation between the quoted price and the trip's actual fare into an acceptance probability. This was needed due to the lack of rejection (due to pricing) data.

$$P(\text{accept} \mid \hat{p}, f) = \frac{1}{1 + \exp\left(\frac{|\hat{p} - f| - rf}{rfs}\right)}$$

- **f**: "actual" fare for the trip
- **p_hat**: predicted price for the trip (in the DQN env this is computed as p_hat = f  multiplier)
- **delta (abs(p_hat - f))** : absolute deviation between quote and baseline
- **r**: max acceptable deviation ratio (default 0.5)
- **delta_max (r*f)**: maximum "acceptable" deviation from the actual fare
- **s**: sigmoid scaling factor ratio, controls how quickly the acceptance probability drops as proposed price gets farther away from the actual fare (default 0.1)

# RESULTS*

| Metric | Static | DQN | Improvement |
|---|---|---|---|
| Total Revenue | $32,316 | $47,608 | +47.32% |
| Acceptance Rate | 81.75% | 99.70% | +17.95pp |
| Avg Price/Ride | $23.46 | $23.88 | +1.8% |
| Accepted Rides | 1,635 | 1,994 | +359 rides |

*Static tested on 2000 rides, DQN tested with 20 episodes * 100 steps/episode.

# MODEL CHOICE TRADE-OFFS

## Alternatives Considered

- **Contextual bandits (LinUCB / Thompson Sampling):** one-step selection, sample-efficient, optimises immediate reward, less suitable for sequential setup.
- **Supervised demand modelling + optimisation (e.g., XGBoost for P(accept | x, price)):** interpretable tabular baseline, depends on sufficient signal/coverage for price-response modelling.
- **Continuous-control RL (DDPG/TD3/SAC) or actor–critic (PPO):** supports fully continuous multipliers but adds tuning/complexity vs discrete actions.

## Why DQN?

- Standard RL baseline with simpler setup than continuous-action RL
- Good test of whether a learned policy beats static pricing before attempting more complicated methods
- Next step: extend to continuous multipliers (SAC/TD3)

# PRACTITIONER FEEDBACK + NEXT STEPS

| Feedback Received | Next Steps |
| --- | --- |
| • **Feedback from a former Uber pricing practitioner:** the project's willingness-to-pay / acceptance modelling is broadly consistent with industry approaches used in ride-hailing pricing. | • **SAC vs DQN** implementation (continuous multipliers)<br>• **Reward function:** current version cuts off at 1.2x; consider a continuous curve<br>• Make reward richer using **profit and cost information**<br>• Layer additional signals into reward shaping (e.g., rain)<br>• Improve **test consistency** (1-to-1 match between static and DQN test sets)<br>• Extend to **driver-side pay offering** |

# FLIGHT DELAY PREDICTION PROJECT

VIEW ON GITHUB

# PROBLEM & SOLUTION

| | |
|---|---|
| Problem | Airlines must forecast departure delays to optimise operational reliability, gate allocation, and crew scheduling. With tens of millions flights and complex interdependencies, statistical forecasting is essential. |
| Solution | Built a forecasting pipeline with domain-driven feature engineering, systematically comparing Logistic Regression, Random Forest, and Gradient Boosting. **Random Forest** emerges as the best performer with AUC 0.73 and CV score 0.74 after hyperparameter tuning and cross-validation. |

# APPROACH

## Domain-Driven Feature Engineering

**Business Ind:** Monday mornings 6–9 AM, corporate travel surge
**Holiday Ind:** Months {1, 6–8, 11–12}, predictable demand surge
**Level 3 Ind:** JFK, LGA, DCA, slot-controlled facilities
**Geospatial features:** Airport latitude/longitude, proximity to congested airspaces
**Aircraft type:** Tail number to FAA model mapping

## Model Development

**Stratified 5-fold cross-validation:** stratified by the target class (delayed/not-delayed), Airline balance is preserved separately via stratified sampling at the data preparation stage

**Three-model comparison:** Logistic Regression (baseline interpretability), Random Forest (non-linearity), Gradient Boosting (performance)

**GridSearchCV tuning:** 27 candidate configurations × 3-fold inner CV

# MODEL CHOICE TRADE-OFFS

## Alternatives Considered

- **Regularised Logistic Regression:** fast + explainable; misses non-linearities/interactions (airport x airline x time-of-day).
- **Gradient Boosting** (XGBoost/LightGBM/CatBoost): often strongest for tabular; higher tuning effort + complexity.
- **Neural nets (MLP):** can model non-linearities; typically needs heavier regularisation and longer iteration cycles.
- **Time-series approaches:** good for temporal dependence; delays are also strongly context-driven.
- **Regression** (minutes delayed): richer output but heavy-tailed/noisy target and harder thresholding.

## Why Random Forest?

- Strong mixed-feature baseline, captures non-linear structure with minimal assumptions
- Works well after encoding, consistent under the same CV setup
- Caveat: can be compute-expensive at this scale

# EARTHQUAKE PREDICTION PROJECT (WIP)

VIEW ON GITHUB

# PROBLEM & SOLUTION

| | |
|---|---|
| Problem | Earthquakes cluster along known fault lines, but their timing and magnitude are hard to predict. Can historical earthquake + fault geological data reveal temporal and spatial patterns for probabilistic forecasting? |
| Solution | Multi-source data pipeline integrating Kandilli Observatory earthquake XML feeds with GEM Global Active Faults Database (GeoJSON), implementing sophisticated **spatial-temporal matching** and building foundation for **Markov chain-based forecasting**. |

# APPROACH

## Technical Highlights

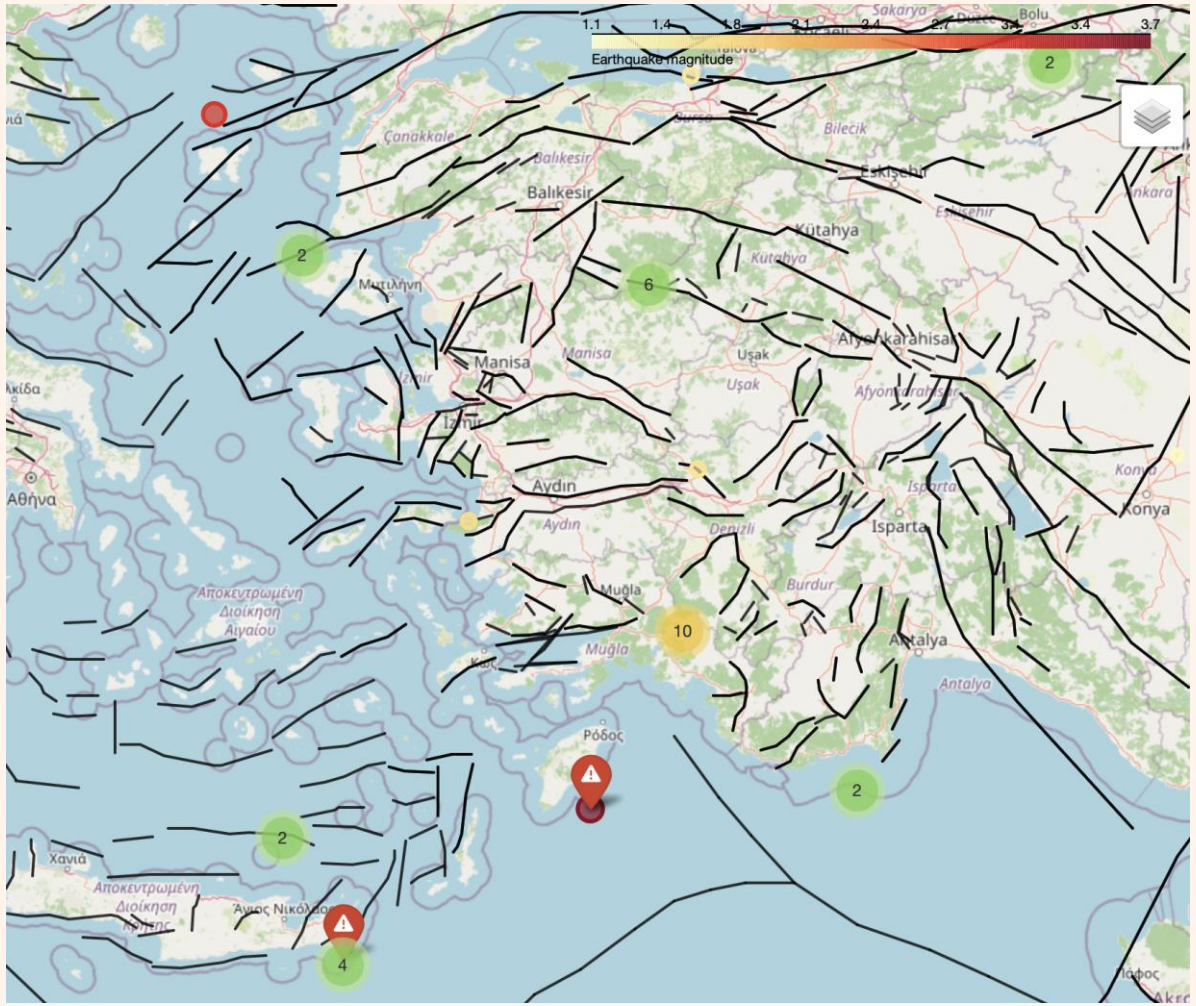
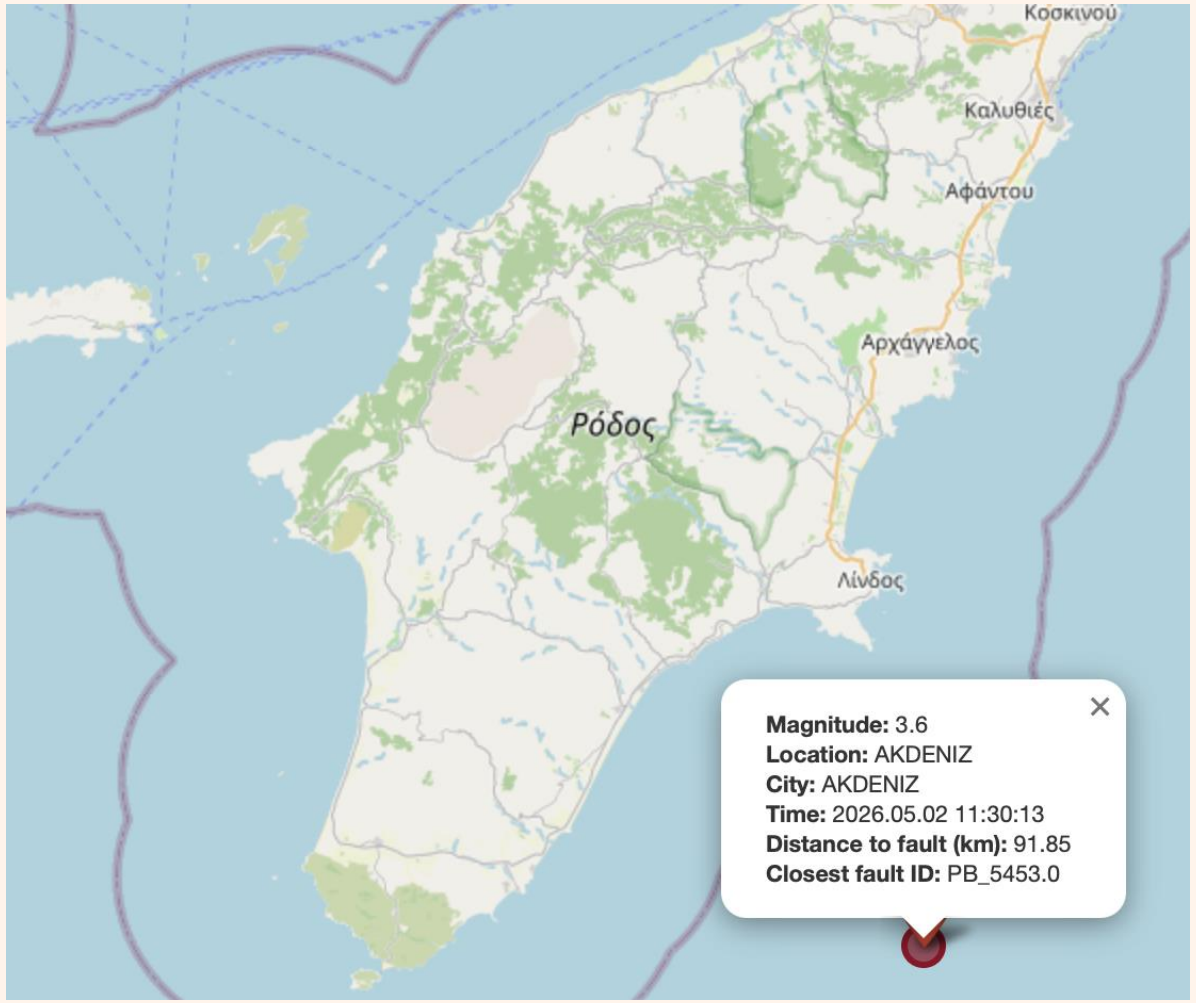
**Earthquake data ingestion:** HTTP download of monthly XML feeds from Boğaziçi University, parsed into structured DataFrames
**Fault database:** GeoJSON from GEM, includes geological properties: dip angle, rake, slip type, seismogenic depth, net slip rate
**Spatial proximity matching:** For each earthquake, find closest fault using SciPy `cdist`
**Multi-radius binning:** Classify earthquakes by proximity
**Interactive visualisation:** Folium maps

## Foundation for Markov Chain Model

**State discretisation:** Each fault occupies a state (Locked, Creeping, Critically Stressed, Ruptured) based on recent seismic activity
**Transition probabilities:** Estimated from historical catalogue, weighted by recency (recent earthquakes have stronger influence on future probability)
**Cumulative feedback:** After each earthquake, fault state updates, reflecting stress accumulation/release cycle
**Spatial coupling (future):** Extended model to account for stress transfer between neighbouring faults (would require the implementation of inter-fault relations)

# BACKTESTING & VALIDATION PLAN

**What is backtesting?**
Commonly used in finance to evaluate trading strategies, it's a historical simulation that repeatedly replays the past to assess how a forecasting method would perform under realistic, time-ordered conditions.

- **Walk-forward backtesting** (time splits): fit using events up to time t, forecast (t, t+Δ], then roll forward (avoid leakage)
- **Define prediction target:** probability of ≥1 event above magnitude threshold within region/fault proximity band over next Δ
- **Compare against baselines:** historical average rate, recency-weighted rate, persistence
- **Use rare-event metrics:** calibration + precision/recall at decision thresholds, lead-time vs hit-rate trade-off
- **Robustness checks:** magnitude cutoffs, horizon length Δ, spatial binning, optional declustering

# INTERACTIVE MAPS